Prerequisites
Create a Foundry project
Create an agent
Configure your data and Foundry IQ
Test the Agent in the playground
Connect to your agent from an app
Test the Integration
Summary
Clean up

Integrate an AI agent with Foundry IQ

In this exercise, you'll use Microsoft Foundry portal to create an agent that integrates with Foundry IQ to search and retrieve information from knowledge bases. You'll create a search resource, configure a knowledge base with sample data, build an agent in the portal, and then connect to it from Visual Studio Code to interact programmatically.

> **Tip**: The code used in this exercise is based on the Microsoft Foundry SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to Microsoft Foundry SDK client libraries for details.

This exercise should take approximately **45** minutes to complete.

> **Note**: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An Azure subscription with permissions to create AI resources
- Visual Studio Code installed on your local machine
- Python 3.13 installed
- Git installed on your local machine
- Basic familiarity with the Microsoft Foundry portal and Python programming

> * Python 3.14 isn't supported yet: some dependencies have no 3.14 build. This lab was tested with Python 3.13.12.

Create a Foundry project

Let's start by creating a Foundry project with the new Foundry experience.

1. In a web browser, open the Foundry portal at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.

   > **Important**: Make sure the **New Foundry** toggle is *On* for this lab to use the updated user interface.

2. Once you toggle to the **New Foundry**, you'll be asked to select a project. In the dropdown, select **Create a new project**.
3. In the **Create a project** dialog, enter a valid name for your project (for example, *agent-iq-lab*).
4. Confirm or configure the following settings for your project:
   - **Foundry resource**: *Create a new Foundry resource or select an existing one*
   - **Subscription**: *Your Azure subscription*
   - **Resource group**: *Create or select a resource group*
   - **Location**: *Select any available region**

   > * Some Azure AI resources are constrained by regional model quotas. In the event of a quota limit being exceeded later in the exercise, there's a possibility you may need to create another resource in a different region.

5. Select **Create** and wait for your project to be created. This may take a few minutes.
6. When your project is created, you'll see the project home page.

Create an agent

1. On the home page, select the **Build** tab, then on the **Agents** tab select **Create agent**.
2. Create your agent with a descriptive name, such as `product-expert-agent`.

When creating an agent, it will deploy the default model (like `gpt-5`). Once your agent is created, you'll see the agent playground with that default model automatically selected for you.

Configure your data and Foundry IQ

Now you'll configure your agent that uses Foundry IQ to search the knowledge base.

1. First, give your agent the following instructions:

```
You are a helpful AI assistant for Contoso, specializing in outdoor camping and hiking
products.
You must ALWAYS search the knowledge base to answer questions about our products or product
catalog. Provide detailed, accurate information and always cite your sources.
If you don't find relevant information in the knowledge base, say so clearly.
```

2. Select **Save** to save your current agent configuration.
3. Then, in the **Knowledge** section, expand the **Add** dropdown, and select **Connect to Foundry IQ**.
4. In the Connect to Foundry IQ window, select **Connect to an AI Search resource** and then **Create new resource** which should open up a dialog to create the resource.
5. Create a search resource with the default settings:
    - **Resource name**: *A globally unique name*
    - **Subscription**: *Your Azure subscription*
    - **Resource group**: *Use the same resource group as your project*
    - **Region**: *The same location as your project*
    - **Pricing tier**: Free *if available, otherwise choose Basic*

> **Important**: If you run into any problems creating the resource here or see a warning message saying **This region is at capacity**, select the link at the bottom of the form to create it from the Azure portal instead.

Now you'll upload sample product information documents to connect to with Foundry IQ.

1. Download the sample product information files by opening a new browser tab and navigating to

```
https://github.com/MicrosoftLearning/mslearn-ai-agents/raw/main/Labfiles/04-integrate-agent-
with-foundry-iq/data/contoso-products.zip
```

2. Extract the files from the zip, which should be 3 PDFs detailing the products from Contoso.
3. Open a new tab and navigate to the Azure portal at `https://portal.azure.com`. In the top search bar, search fo **Storage accounts** and select **Storage accounts** from the services section.
4. Create a storage account with the following settings:
    - **Subscription**: *Your Azure subscription*
    - **Resource group**: *Use the same resource group as your project*
    - **Storage account name**: *A unique storage account name*
    - **Region**: *The same location as your project*
    - **Primary service**: *Azure Blob Storage or Azure Data Lake Storage*
    - **Performance**: *Standard*
    - **Redundancy**: *Locally-redundant storage (LRS)*
5. Once created, go to the storage account you created and select **Upload** from the top bar.
6. In the **Upload blob** blade, create a new container named `contosoproducts`.
7. Browse for the files extracted from the zip file, select all 3 PDF files, and select **Upload**.
8. Once your files are uploaded, navigate to the search service you created.

9. On the left pane, under **Security + networking** > **Keys**, select **Both** for API Access control and confirm the selection. Once complete, leave the Azure Portal tab open and navigate back to the Foundry portal tab and refresh the page.
10. Verify you are on the **Knowledge** page, select **Create a knowledge base**, choosing **Azure Blob Storage** as your knowledge source, then select **Connect**.
11. Configure your knowledge source with the following settings:
    - **Name**: `ks-contosoproducts`
    - **Description**: `Contoso product catalog items`
    - **Storage account name**: *Select your storage account*
    - **Container name**: `contosoproducts`
    - **Authentication type**: *API Key*
    - **Content extraction mode**: *minimal*
    - **Embedding model**: *Select the available deployed model, likely text-embedding-3-small*
    - **Chat completions model**: *Select the available deployed model, likely gpt-5*
12. Select **Create**.
13. On the knowledge base creation page, select the `gpt-5` model from the **Chat completions model** dropdown, leaving the rest of the field defaults as is.
14. Select **Save knowledge base**, and then refresh your browser to verify the knowledge source status is *active*. If it isn't yet, wait a minute and refresh your page until it is.
15. Select the back button to return to the **Knowledge** page, then select the **Manage** link next to the *Connection* drop-down.
16. Scroll down to the **Connected resources**, where you should see your search service. Select that row, find the **Authentication** section.
17. Select **Key authentication** and then select **Edit authentication**.
18. Leaving the dialog open, return to the Azure portal tab which should still be on your search service **Keys** page. Copy one of those keys into the dialog in Foundry and select **Save**.

Your Foundry IQ settings should now be complete.

Test the Agent in the playground

Before connecting from code, test your agent in the portal playground.

1. Navigate back to your agent on the **Build** > **Agents** page, and select the agent you created.
2. In the agent page, you should see a playground tab selected. Find the knowledge section and add Foundry IQ, selecting the connection and knowledge base you created.
3. Try the following test queries to verify the agent can retrieve information from the knowledge base:
    - `What types of tents does Contoso offer?`
    - `Tell me about which backpacks are available in XL.`
    - `What camping accessories are available?`
4. Review the responses and notice:
    - The agent provides specific information from the knowledge base
    - Citations or references to the source documents may be included
    - The agent stays focused on product information
5. You can also try interacting with your agent in the **Preview agent** for a more refined webapp experience.
6. In the agent details page, locate and copy the following information to a notepad (you'll need these later):
    - **Agent name**: This is the name you created (`product-expert-agent`)
    - **Project endpoint**: Found in the project settings or home page

Configure the agent to require approval for tool calls

When you create an agent in the portal, its Foundry IQ (knowledge) tool runs **without** asking for approval by default. To ensure your app can review and control each knowledge base lookup, you'll change the agent to require approval before it uses tools with the Foundry Toolkit for VS Code extension.

> **Note**: The Foundry portal doesn't currently expose a setting to change this approval behavior, so you'll configure it from the Foundry Toolkit extension instead.

1. In Visual Studio Code, select **Extensions** from the left pane (or press **Ctrl+Shift+X**), then search the marketplace for the `Foundry Toolkit for VS Code` extension from Microsoft and select **Install** (if it isn't already installed).

   > **Note**: The extension is currently listed as **Foundry Toolkit**, but some VS Code labels, commands, or older screenshots may still refer to **AI Toolkit**. In this lab, treat those names as referring to the same extension experience.

2. Select the **Foundry Toolkit** icon in the sidebar, and sign in to your Azure account if you're prompted.

   > **Note**: If you're unable to sign in with the Foundry Toolkit extension, you my need to select the Azure extension. Sign in there, then navigate back to the Foundry Toolkit to access your resources.

3. Under **Microsoft Foundry Resources**, choose **Set Foundry Project** and select the project you created earlier.
4. Select the **Agents** section. Under **Prompt Agents** tab on the page opened, select your `product-expert-agent` agent to open the **Agent Builder** window.
5. In the **Tools** section, you should already see a tool named with a `kb-knowledgebase` prefix followed by a unique ID (for example, `kb-knowledgebase677-7w5fj`). This is the Foundry IQ knowledge base tool, and it was added automatically when you connected Foundry IQ in the portal.

   > **Note**: The agent lists more than one tool. The Foundry portal adds a **Web search** tool to new agents by default, and you may also see a standalone **Azure AI Search** tool. The agent actually calls the `kb-knowledgebase...` tool when it searches your knowledge base, so setting approval on any other tool has no effect.

6. Select the ellipsis (**...**) icon on the `kb-knowledgebase...` tool, then select **Configure** and select the **Never auto-approve tools**, and save your changes, on the top of the playground panel select **Deploy** to save your changes.

Your agent will now request approval each time it uses Foundry IQ to search the knowledge base, which the client app you complete next will handle.

Connect to your agent from an app

Now you'll create a Python application to interact with your agent programmatically. Starter files have been provided in the GitHub repository to help you get started quickly.

Prepare to develop an app in Visual Studio Code

Now let's use Visual Studio Code to develop an app. The code files for your app have been provided in a GitHub repo.

1. Start Visual Studio Code, and open the command palette (Shift+Ctrl+P). Then search for and run the **Git: Clone** command to clone the `https://github.com/MicrosoftLearning/mslearn-ai-agents` repo to a local folder (it doesn't matter which folder).
2. When the repository has been cloned, open the folder in Visual Studio Code.

   > **Note**: If Visual Studio Code shows you a pop-up message prompting you to trust the code you are opening, click **Yes, I trust the authors** option to continue.

3. Wait while additional files are installed to support the Python code projects in the repo (if prompted).

> **Note**: If you are prompted to install required assets to build and debug, select **Not Now**.

4. In the **Explorer** pane, expand the **Labfiles/04-integrate-agent-with-foundry-iq/Python** folder.

   The provided files include application code, configuration settings, and the agent client starter code.

Configure the application settings

1. In Visual Studio Code, in the **Labfiles/04-integrate-agent-with-foundry-iq/Python** folder, open the **.env** configuration file.
2. In the code file, replace the **your_project_endpoint** placeholder with the endpoint for your project (copied from the project **Home** page in the Foundry portal) and ensure that the AGENT_NAME variable is set to your agent name (which should be *product-expert-agent*).
3. After you've replaced the placeholder, save the file.

Complete the agent client code

> **Tip**: As you add code, be sure to maintain the correct indentation. Use the comment indentation levels as a guide.

1. In Visual Studio Code, in the **Labfiles/04-integrate-agent-with-foundry-iq/Python** folder, open the **agent_client.py** code file.
2. Review the starter code that has been provided, including:
   - Import statements and configuration loading
   - The `send_message_to_agent()` function structure
   - The `display_conversation_history()` function
   - The main program loop
3. Find the first **TODO** comment and add the following code to connect to the project, get the OpenAI client, retrieve the agent, and create a new conversation:

> **Tip**: Be careful to maintain the correct indentation level.

Code Copy

```
# Connect to the project and agent
credential = DefaultAzureCredential(
    exclude_environment_credential=True,
    exclude_managed_identity_credential=True
)
project_client = AIProjectClient(
    credential=credential,
    endpoint=project_endpoint
)

# Get the OpenAI client
openai_client = project_client.get_openai_client()

# Get the agent
agent = project_client.agents.get(agent_name=agent_name)
print(f"Connected to agent: {agent.name} (id: {agent.id})\n")

# Create a new conversation
conversation = openai_client.conversations.create(items=[])
print(f"Created conversation (id: {conversation.id})\n")
```

4. Find the second **TODO** comment inside the `send_message_to_agent()` function and add the following code to send messages and handle responses, including MCP approval requests:

Code Copy

```
# Add user message to the conversation
openai_client.conversations.items.create(
    conversation_id=conversation.id,
    items=[{"type": "message", "role": "user", "content": user_message}],
)

# Store in conversation history (client-side)
conversation_history.append({
    "role": "user",
    "content": user_message
})

# Create a response using the agent
response = openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},
    input=""
)

# Loop until a response has no pending approval requests (zero, one, or many)
while True:
    approval_requests = [
        item for item in (getattr(response, "output", None) or [])
        if getattr(item, "type", None) == "mcp_approval_request"
    ]

    if not approval_requests:
        break

    approval_items = []
    for approval_request in approval_requests:
        print(f"[Approval required for: {approval_request.name}]\n")
        print(f"Server: {approval_request.server_label}")

        # Show the tool call arguments for transparency
        import json
        try:
            args = json.loads(approval_request.arguments)
            print(f"Arguments: {json.dumps(args, indent=2)}\n")
        except Exception:
            print(f"Arguments: {approval_request.arguments}\n")

        approval_input = input("Approve this action? (yes/no): ").strip().lower()
        approved = approval_input in ['yes', 'y']
        print("Approving action...\n" if approved else "Action denied.\n")

        approval_items.append({
            "type": "mcp_approval_response",
            "approval_request_id": approval_request.id,
            "approve": approved
        })

    # Send the approval decisions and fetch the next response
    openai_client.conversations.items.create(
        conversation_id=conversation.id,
        items=approval_items
    )
```

```
response = openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},
    input=""
)
```

> **Note**: The agent doesn't always request approval, and it can occasionally request approval for more than one tool call in the same turn. Looping until `approval_requests` is empty handles both cases correctly.

5. After you've added the code, save the file.
6. Review the code now uses the conversations API to manage interactions with your agent, where:
    - A conversation is created and tracked by its ID
    - User messages are added to the conversation using `conversations.items.create()`
    - Responses are generated using `responses.create()` with an agent reference
    - **MCP approval handling**: When the agent needs to access Foundry IQ, it requests approval by returning one or more `mcp_approval_request` items in the response output
    - The code loops, prompting you to approve or deny each pending request, until the agent returns a response with no outstanding approval requests (including the case where no approval was ever needed)
    - After each approval/denial, an `mcp_approval_response` is added to the conversation and a new response is generated
    - The agent retrieves information from Foundry IQ based on your approval decision

Test the Integration

Now you'll run your application and test the agent's ability to retrieve information from the knowledge base.

1. In Visual Studio Code, open an integrated terminal for the **Labfiles/04-integrate-agent-with-foundry-iq/Python** folder by right-clicking the folder and selecting **Open in Integrated Terminal**.

2. First, create a virtual environment and install dependencies.

Code Copy

```
python -m venv labenv
./labenv/Scripts/activate
pip install -r requirements.txt
```

3. In the terminal pane, enter the following command to sign into Azure.

Code Copy

```
az login
```

> **Note**: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *–tenant* parameter. See Sign into Azure interactively using the Azure CLI for details.

4. When prompted, follow the instructions to open the sign-in page in a new tab and enter the authentication code provided and your Azure credentials. Then complete the sign in process in the command line, selecting the subscription containing your Foundry resource if prompted.

5. In the terminal pane, run your application:

```
python agent_client.py
```

6. When the application starts, test the agent with the following queries:

   Query 1 - Product Categories:

   ```
   What types of outdoor products does Contoso offer?
   ```

   When prompted for approval, type **yes** to allow the agent to search the knowledge base. Observe how the agent retrieves information from multiple documents in the knowledge base.

   Query 2 - Specific Product Details:

   ```
   Tell me about the weatherproof features of your tents.
   ```

   Approve the request and notice how the agent provides specific details from the tents catalog.

   Query 3 - Product Comparisons:

   ```
   What's the difference between your daypacks and expedition backpacks?
   ```

   Approve the request and see how the agent can synthesize information from the backpacks guide.

   Query 4 - Accessories and Add-ons:

   ```
   What camping accessories would you recommend for a weekend hiking trip?
   ```

   Approve the request and observe the agent's ability to provide recommendations based on the knowledge base.

   Query 5 - Follow-up Question:

   ```
   How much do those items typically cost?
   ```

   Notice how the agent maintains conversation context from your previous query.

7. Type `history` to view the complete conversation history.
8. Type `quit` when you're done testing.

Review the results

Consider the following aspects of the agent's responses:

- **MCP Approval Flow**: Each time the agent needs to access the knowledge base, it requests approval, giving you control over external tool usage
- **Accuracy**: The agent provides information directly from the knowledge base documents

- **Citations**: The agent may include source references or document IDs
- **Context awareness**: The agent remembers previous messages in the conversation
- **Grounding**: The agent indicates when it cannot find relevant information in the knowledge base
- **Error handling**: The application gracefully handles errors and connection issues

Summary

In this exercise, you:

- Created a Foundry project and agent with the new Foundry UI
- Built a knowledge base with product information documents
- Configured an agent in the portal with Foundry IQ enabled
- Connected to your agent from Visual Studio Code using the Python SDK
- Implemented a client application with MCP approval handling, conversation history, and error handling
- Tested the agent's ability to retrieve and synthesize information from the knowledge base with user-controlled approval for external tool access

This demonstrates how to integrate AI agents with Foundry IQ to create intelligent applications that can search and retrieve information from enterprise knowledge bases while maintaining conversational context.

Clean up

If you've finished exploring Azure AI Agent Service and Foundry IQ, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

1. In a web browser, open the Azure portal at `https://portal.azure.com`.
2. Navigate to the resource group containing your Foundry resource and AI Search resources.
3. On the toolbar, select **Delete resource group**.
4. Enter the resource group name and confirm that you want to delete it.